

Lab: 0100

Half Adder, Full Adder & 4-Bit Ripple Adder

Adders

*Compose half adders into full adders. Cascade four full adders.
Watch the carry ripple.*

CSA222: Modern Computer Architecture

What You'll Build Today

Three modules. Each one composes the previous. By the end, you'll add two 4-bit numbers in silicon.

01

Half Adder

Two inputs A, B. Produces Sum and Cout. The simplest 1-bit add — but it can't accept a carry-in.

02

Full Adder

Three inputs A, B, Cin. Built from two half adders + an OR gate. The true 1-bit adder cell.

03

4-Bit Ripple Adder

Four full adders chained. Cout of stage i becomes Cin of stage $i+1$. Adds two 4-bit numbers.

Why Adders Matter

Addition is the atom of arithmetic. Every CPU instruction that does math leans on one of these cells.

Subtraction

$A - B$ is just $A + (\sim B) + 1$ — done with the same adder, with $C_{in} = 1$ and one input inverted.

Multiplication

$A \times B$ is a sequence of shifts and adds. Multipliers are arrays of adders.

Program counters

$PC \leftarrow PC + 4$ happens every clock cycle. Always an adder. Address arithmetic too.

Master a 1-bit adder, and you've cracked open everything that follows: subtractors, multipliers, ALUs, the entire datapath.

BINARY ADDITION · $0 + 0 = 00$

$0 + 1 = 01$

$1 + 0 = 01$

$1 + 1 = 10$

Half Adder · The Equations

Two 1-bit inputs. Two 1-bit outputs. Two Boolean equations. That's the whole device.

SUM

$$\text{Sum} = A \oplus B$$

Sum is 1 when A and B differ. That's XOR.

CARRY-OUT

$$\text{Cout} = A \cdot B$$

Carry-out is 1 only when both are 1. That's AND.

WHY "HALF"?

A half adder has no place to accept a carry from a previous stage. That's only half of what real binary addition needs — so you can't chain them. The full adder, two slides later, fixes this.

Half Adder · Truth Table & Schematic

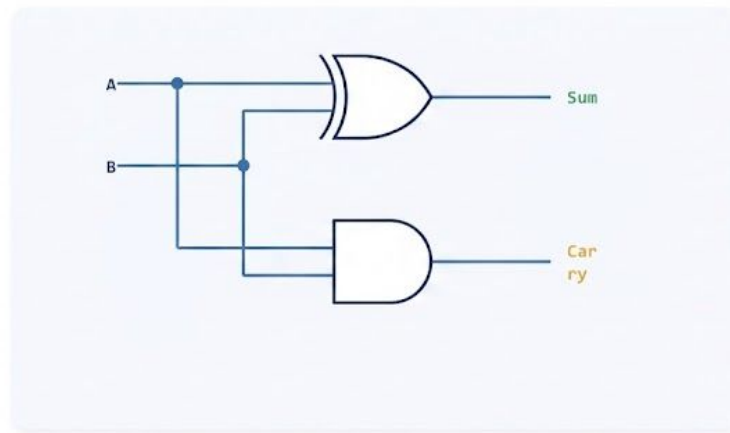
Four input rows. Two outputs each. Read it left-to-right.

TRUTH TABLE

A	B	Sum	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Last row · Sum = 0, Cout = 1. That's the carry: $1+1 = 10$ in binary.

SCHEMATIC



Half Adder · Verilog Implementation

Two continuous assignments. No always blocks, no clocks. Pure combinational.

```
half_adder.v
// 1-bit half adder
module half_adder (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire cout
);

    assign sum  = a ^ b;    // XOR
    assign cout = a & b;    // AND

endmodule
```

NOTES

^ bitwise XOR. & bitwise AND.

Two assign statements run in parallel — there's no ordering.

Both outputs settle the moment a or b changes.

In real silicon, this is one XOR cell and one AND cell, side by side.

Half Adder · Self-Checking Testbench

Four input combinations. Compare to expected. Print PASS or FAIL for each row of the truth table.

```
ha_tb.v
module ha_tb;
    reg a, b; wire sum, cout;
    half_adder dut(.a(a), .b(b),
        .sum(sum), .cout(cout));

    task check(input ea,eb,es,ec);
        begin a=ea; b=eb; #1;
            $display("%s a=%b b=%b s=%b c=%b",
                (sum===es && cout===ec)?"PASS":"FAIL",
                a, b, sum, cout);
        end
    endtask

    initial begin
        $dumpfile("ha.vcd"); $dumpvars(0,ha_tb);
        check(0,0,0,0); check(0,1,1,0);
        check(1,0,1,0); check(1,1,0,1);
        $finish;
    end
endmodule
```

EXPECTED OUTPUT

PASS a=0 b=0 s=0 c=0

PASS a=0 b=1 s=1 c=0

PASS a=1 b=0 s=1 c=0

PASS a=1 b=1 s=0 c=1

*Four lines, all PASS. If any FAIL appears —
check operator precedence in your assigns.*

Why a Half Adder Isn't Enough

Try to add two multi-bit numbers and you'll hit a wall on the very second column.

ADD 11 + 01

```
  1 1    ← A
+ 0 1    ← B
-----
 1 0 0
```

Bit 0: $1 + 1 \rightarrow \text{sum}=0, \text{carry}=1$ ✓

Bit 1: $1 + 0 + \text{carry} \rightarrow \text{needs 3 inputs!}$

THE PROBLEM

Bit position 1 has three signals to sum:

A[1]

B[1]

Carry-in from bit 0

A half adder has only two inputs. It cannot accept that carry. We need a 3-input adder cell — the full adder.

Full Adder · The Equations

Three 1-bit inputs (A, B, Cin) summed into a 2-bit result (Cout, Sum).

SUM

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

Three-way XOR. Sum is 1 when an odd number of inputs are 1.

CARRY-OUT

$$\text{Cout} = (A \cdot B) + (\text{Cin} \cdot (A \oplus B))$$

Carry when both A,B are 1 — or when the partial sum ($A \oplus B$) is 1 and Cin is also 1.

KEY INSIGHT

Look at the Sum formula: it's a chain of XORs. And the Cout formula has ($A \oplus B$) baked into it. That shared ($A \oplus B$) term is the hint — you can build a full adder from two half adders.

Full Adder · Truth Table

Eight rows — all 2^3 combinations of (A, B, Cin). Sum is 1 on odd-count rows; Cout is 1 when two or more inputs are 1.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

READING THE TABLE

Sum

1 when an odd number of inputs (1 or 3) are 1. Pure parity function.

Cout

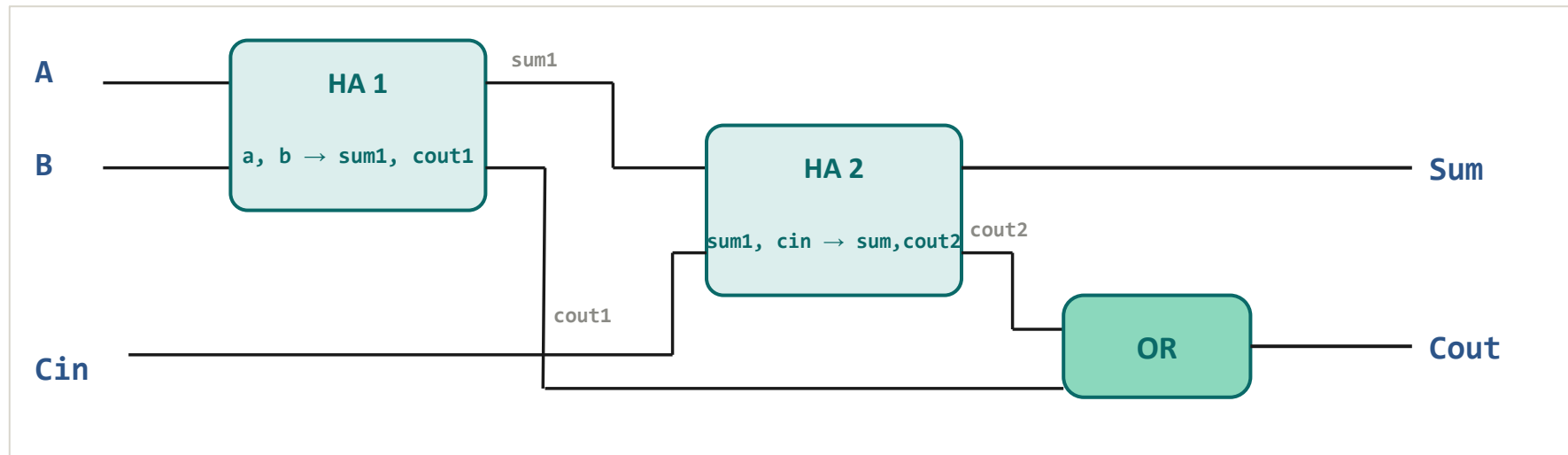
1 when two or more inputs are 1. Pure majority function.

Sum + Cout

Together they encode the 2-bit count of how many inputs are 1.

Full Adder · Built From Two Half Adders

Stage 1 adds A and B. Stage 2 adds that result to Cin. An OR gate combines the two carry signals.



Sum from HA2 is the final sum. Carry from either HA is the final carry — that's why OR works.

Why the OR Gate Works for the Carry-Out

The two half-adder carries can never be 1 at the same time. So OR is enough — no third AND is needed.

01

$$\text{cout1} = A \cdot B$$

First half adder sees A and B. Its carry is 1 only when both A and B are 1.

02

$$\text{cout2} = \text{sum1} \cdot \text{Cin}$$

Second half adder sees $\text{sum1} = (A \oplus B)$ and Cin. Its carry is 1 when sum1 and Cin are both 1.

03

They can't both be 1

If $A=B=1$, then $\text{sum1} = A \oplus B = 0$, so $\text{cout2} = 0$. cout1 and cout2 are mutually exclusive — OR or XOR both work.

ALGEBRA · $\text{Cout} = \text{cout1} + \text{cout2} = AB + (A \oplus B) \cdot \text{Cin} \leftrightarrow$ *the standard full-adder carry equation.*

Full Adder · Verilog (Structural)

Instantiate two half_adder modules, OR their carries together. No new assigns — pure composition.

```
full_adder.v
// 1-bit full adder · 2 HAs + 1 OR
module full_adder (
    input  wire a, b, cin,
    output wire sum,
    output wire cout
);
    wire s1, c1, c2;

    half_adder ha1 (.a(a), .b(b),
                   .sum(s1), .cout(c1));

    half_adder ha2 (.a(s1), .b(cin),
                   .sum(sum), .cout(c2));

    assign cout = c1 | c2; // OR
endmodule
```

WHAT'S WIRED

ha1 sees a, b directly. Its sum is the partial sum s1.

ha2 sees s1 and cin. Its sum becomes the final Sum.

c1, *c2* are the two HA carry-outs.

$\text{cout} = \text{c1} \mid \text{c2}$ combines them — exactly one (or neither) is ever 1.

Full Adder · Testbench (All 8 Cases)

Pack three input bits at once — same {a, b, cin} = i[2:0] trick you used for the MUX testbench in Lab 2.

```
fa_tb.v
module fa_tb;
  reg a, b, cin; wire sum, cout;
  full_adder dut(.a(a), .b(b), .cin(cin),
                .sum(sum), .cout(cout));

  integer i;
  initial begin
    $dumpfile("fa.vcd"); $dumpvars(0, fa_tb);
    for (i = 0; i < 8; i = i + 1) begin
      {a, b, cin} = i[2:0]; #5;
      $display("a=%b b=%b cin=%b → sum=%b cout=%b",
              a, b, cin, sum, cout);
    end
    $finish;
  end
endmodule
```

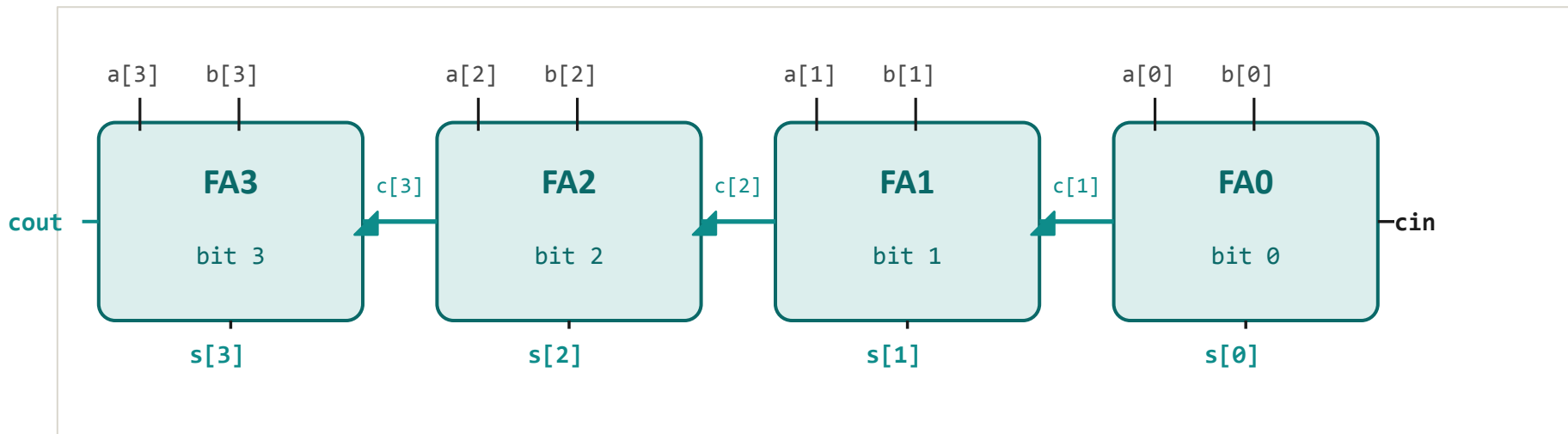
RUN IT

```
$ iverilog -o fa.vvp \
  half_adder.v \
  full_adder.v \
  fa_tb.v
$ vvp fa.vvp
$ gtkwave fa.vcd
```

Pass all three .v files. Verilog won't find half_adder otherwise.

4-Bit Ripple-Carry Adder · The Idea

Chain four full adders. Bit 0 on the right, bit 3 on the left. Each carry-out feeds the next carry-in.



"Ripple" = the carry has to travel through every stage before the last sum is correct. Each stage waits for the previous.

4-Bit Ripple Adder · Verilog

Four full_adder instances. One internal 3-bit carry wire. Connect `cout[i]` to `cin[i+1]`.

```
adder4.v
module adder4 (
    input wire [3:0] a, b,
    input wire      cin,
    output wire [3:0] sum,
    output wire      cout
);
    wire c1, c2, c3; // internal carries

    full_adder fa0(.a(a[0]),.b(b[0]),.cin(cin),
                  .sum(sum[0]),.cout(c1));
    full_adder fa1(.a(a[1]),.b(b[1]),.cin(c1),
                  .sum(sum[1]),.cout(c2));
    full_adder fa2(.a(a[2]),.b(b[2]),.cin(c2),
                  .sum(sum[2]),.cout(c3));
    full_adder fa3(.a(a[3]),.b(b[3]),.cin(c3),
                  .sum(sum[3]),.cout(cout));
endmodule
```

KEY POINTS

`[3:0]` declares a 4-bit vector. `a[0]` is the LSB.

`c1, c2, c3` are internal — only carry-in and carry-out are exposed.

`cin` → `fa0`, and `fa3.cout` → `cout`. The chain is complete.

Try generate loops in the stretch problems for an N-bit version.

4-Bit Ripple Adder · Testbench Vectors

Four targeted test cases. Each one stresses a different aspect of carry behaviour.

```
adder4_tb.v
module adder4_tb;
    reg [3:0] a, b; reg cin;
    wire [3:0] sum; wire cout;
    adder4 dut(.a(a),.b(b),.cin(cin),
               .sum(sum),.cout(cout));

    initial begin
        $dumpfile("adder4.vcd");
        $dumpvars(0, adder4_tb);
        cin = 0;
        a=4'b0001; b=4'b0001; #10; // 1+1=2
        a=4'b0111; b=4'b0001; #10; // 7+1=8
        a=4'b1111; b=4'b0001; #10; // overflow
        a=4'b1000; b=4'b1000; #10; // 8+8=16
        $finish;
    end
endmodule
```

WHY THESE 4?

0001 + 0001 smallest carry — just bit 0 to bit 1.

0111 + 0001 carry ripples through bits 0, 1, 2.

1111 + 0001 carry ripples through ALL four bits — overflow out.

1000 + 1000 one-bit add, immediate overflow to cout.

Walking Through Test Cases · Part 1

Trace each carry by hand. If your simulation disagrees, you have a bug in the adder — not in the testbench.

$$0001 + 0001 = 00010$$

```
  0 0 0 1  ← a
+ 0 0 0 1  ← b
-----
  0 0 1 0  sum = 0010
cout = 0
```

$$0111 + 0001 = 01000$$

```
  0 1 1 1  ← a
+ 0 0 0 1  ← b
-----
  1 0 0 0  sum = 1000
cout = 0
```

Bit 0: $1+1 \rightarrow s=0, c=1$. Bit 1: $0+0+1 \rightarrow s=1, c=0$. No further carry.

Carry travels: bit 0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3, and bit 3 finally outputs $s=1, c=0$.

Walking Through Test Cases · Part 2

Now the edge cases — when 4 bits are too few to hold the result, the final carry-out tells you so.

$1111 + 0001 = 10000 \leftarrow \text{overflow}$

```
  1 1 1 1   ← a
+ 0 0 0 1   ← b
-----
  0 0 0 0   sum = 0000
cout = 1    ← overflow!
```

Carry ripples through ALL four stages. sum looks like zero, but cout=1 means "there's a fifth bit".

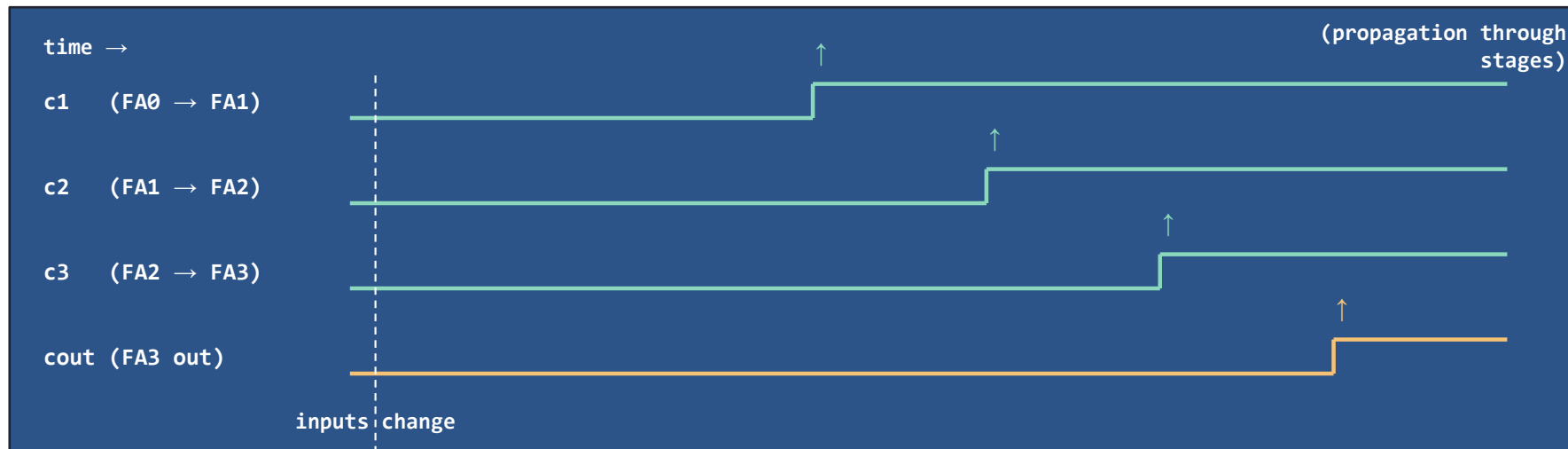
$1000 + 1000 = 10000$

```
  1 0 0 0   ← a
+ 1 0 0 0   ← b
-----
  0 0 0 0   sum = 0000
cout = 1
```

Only bit 3 has a 1+1 → carry out immediately. No rippling, but still an overflow.

Watching the Carry Ripple in GTKWave

Open adder4.vcd and add a, b, sum, cout, c1, c2, c3 to the waveform. Zoom in on the 1111 + 0001 transition.



Each carry rises slightly later than the previous one — that staircase is the "ripple" propagating in real time.

Why Ripple-Carry is Slow

In real silicon, every gate has a delay. The carry has to walk through all of them — serially.

~2

gate delays per FA stage

Each full adder adds 2 levels of logic between its Cin and its Cout.

8

delays for a 4-bit add

4 stages $\times$ 2 delays = 8. Doable. But scale up...

128

delays for a 64-bit add

64 stages $\times$ 2 delays. Modern CPUs can't tolerate this latency.

THE FIX

Compute every carry in parallel using the equation $Cout = G + (P \cdot Cin)$, where $G = A \cdot B$ and $P = A \oplus B$. That's the carry-look-ahead adder — Lab 4.

Common Errors · And the Fix

Five traps specific to this lab. Spot them quickly — save yourself 30 minutes of waveform-staring.

- `cout` always 0, even when `1+1+1`

Probably wrote `cout = c1 & c2` instead of `c1 | c2`. AND only fires when BOTH carries are 1, which never happens.

- `sum` bits in reverse order

`a[0]` is the LSB. If you wired `sum[3]` to `fa0`, you've reversed everything. Double-check your indices.

- Last `sum` is always X (red)

`fa3` has no `Cin` connection — you forgot to wire `c3` in. Unassigned Verilog inputs default to X (unknown).

- Waveform shows correct `sum`, but at time 0

You assigned all inputs at `t=0` with no `#delay`. Add small delays between vectors so transitions are visible.

Take These Home

Three stretch problems. Solutions discussed at the start of Lab 4.

01

Build it from one half adder

Refactor `full_adder.v` to use only ONE `half_adder` instance plus a few primitive gates. Verify it still passes all 8 cases.

02

Parametric N-bit adder

Rewrite `adder4.v` as a parameterised `adder#(N)` module with a Verilog generate loop. Test with $N = 8$ and $N = 16$.

03

Overflow detection

Add an overflow output to your 4-bit adder using signed arithmetic semantics — $V = c3 \oplus c4$ (carry into MSB XOR carry out).

Pick one. All three if you're feeling sharp.

Why This Matters

The 1-bit full adder is the building block. Every arithmetic structure in a CPU starts here.

TODAY

You built an ALU's most basic operation from primitive gates. Two HAs, four FAs, one VCD trace.

TOMORROW

Lab 4 trades depth for width — carry-look-ahead computes every carry in parallel. Faster, more transistors.

REAL CHIPS

Modern CPUs use carry-select, Kogge-Stone, or Brent-Kung adders. All of them still decompose into FA cells like the one you just wrote.

BEYOND

Multipliers are arrays of FAs. Floating-point units are wider FAs. Even GPUs and TPUs lean on these cells billions of times per second.

You just built, in 90 minutes, the cell that ships in every CPU on Earth.

END OF LAB 03

Two half adders.
One full adder.
Four cells, one ALU's heart.

Questions?



**Thanks
for
watching!**